

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE CODE: CSA1441 **COURSE NAME:** Compiler Design for Engineers

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)

Assessment Tool Weightage: 20%

QUESTIONS:

Total Marks:50

Mode: Take-Home

Hours : 1 Day

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

*I M Harinath/192425020 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Q. No	Error Identification (20)	Root Cause Analysis (30)	Correction & Justification (25)	Application of Concepts (15)	Clarity & Presentation (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 30	/ 25	/ 15	/ 10	/ 100

C01 - Error Analysis Table (AT-2)

(a)

Full Compiler Phase Error analysis

(a) Identify all errors with description

Error	Description
Missing: after float $x = 3.5$	Syntax error
Missing: after $y = num1 + 10$	Syntax error
Missing:) in if ($y = 35$)	Syntax error
Missing: after print () arr[3]	Syntax error Runtime
*p = 20;	uninitialized Pointer
10/0	Division by zero (Runtime)

(b) Classification

- Lexical: None
- Syntax: missing Semicolons, Parenthesis
- Semantic: Division by zero
- Logical: None
- Runtime: Array out of bounds, Pointer misuse.

(c) Explain which errors are detected during Compilation and which occurs at runtime.

Compiler time: missing Semicolons, Parenthesis.

Runtime: Array overflow, Pointer misuse, Division by zero.

② Rewrite the Corrected version of Program.

```
#include <stdio.h>
int main()
{
    int num1 = 05;
    float x = 3.5;
    int y;
    y = num1 + 10;
    if (y == 35) {
        printf("Correct value\n");
    }
    int arr[3] = {1, 2, 3};
    printf("%d\n", arr[2]);
    int value = 20;
    int *p = &value;
    printf("%d\n", *p);
    while (num1 < 30)
    {
        printf("%d\n", num1);
        num1++;
    }
    return 0;
}
```

③ Analyze the impact of division by zero and Pointer misuse.

- Division by zero: Program crashes or produces undefined behaviors.

- Pointer misuse: Causes Segmentation fault for memory Corruption.

④ Suggest improvements in Compiler design for better error detection.

- Better syntax error messages.
- Detect array bound violations
- warn about uninitialized pointers
- warn for division by zero.

- ② Advanced Lexical Error Identification
- ① identify all lexical errors with invalid Token.

invalid Token

5num
valuek
10#20
data@
"Sample program"
0x112
3.4.5
"Hello"

valid Token

num5
value
10+20
data
"Sample program"
0x12
3.45
"Hello"

- ⑤ Explain which lexical rule is ~~not~~ violated.

- Identifier starts with digit
- invalid Symbols
- invalid hexadecimal Constant
- invalid Floating Constant
- unterminated String literal.

③ invalid

5num
valuek
10#20
data@
3.4.5
"Hello"

valid Tokens

num5
value
10+20
data
3.45
"Hello"

④ Explain how a lexical analyzer processes such error
The lexical analyzer scans characters and converts them into tokens. Invalid tokens are reported as lexical errors.

⑤ Design DFA rule to detect invalid identifiers starting with digits.

Identifier must start with
letter or $-$

↓
letters, digits or $-$

↓
valid identifier

If first character is a digit $\rightarrow$ Reject.

⑥ Suggest improvements in lexical error reporting.

- Highest line numbers.
- Suggest corrected token.
- Display meaningful error messages.

⑦ Intermediate Code and Optimization Analysis

① Identify all errors and inefficiencies in program.

- Repeated Computation $a * b$
- If $(a = 5)$ uses assignment instead of Comparison.
- $arr[2]$ is out of bounds
- uninitialized Pointer

② Classify errors according to Compiler phases.

- Syntax: None.
- Semantic: assignment inside condition.
- Runtime: array overflow, pointer misuse.
- Optimization: Repeated expressions.

③ Generate three address code (TAC) for exp.

$$t_1 = a * b$$

$$t_2 = t_1 + t_1$$

$$t_3 = t_2 + t_1$$

$$c = t_3$$

④ Apply DAG representation to eliminate redundant Computations:

$$R_1 = a$$

$$R_2 = b$$

$$\text{MUL } R_1, R_2$$

$$\text{ADD } R_1, R_1$$

$$\text{ADD } R_1, R_2$$

⑤ DAG optimization

$$a * b$$

⑥ Analyze runtime impact of array bounds violation and Pointer misuse.

- array overflow accesses invalid memory
- pointer misuse causes segmentation fault

⑦ Error Identification and Classification

① Identify all errors in the program.

missing semicolon after

in $x = 0$

② Classify each error as lexical, syntax, semantic, logical or runtime.

Error: missing semicolon (;) after $\text{int } x = 10$

Classification: syntax error

No errors in lexical, semantic, logical, runtime.

② Rewrite the Corrected Program.

```
#include <stdio.h>
int main()
{
    int x = 10;
    int y = 20;
    int z = x + y;
    printf("%d\n", z);
    return 0;
}
```

③ Compiler Error analysis

① Identify the runtime errors in program.

- Array index out of bounds

arr[3]

- uninitialized pointer.

*ptr = 50;

② Explain why these errors occur

- Array has only two elements

- pointer does not point to valid memory.

③ Rewrite the Corrected program.

```
#include <stdio.h>
int main()
{
    int arr[2] = {5, 10};
    printf("%d\n", arr[1]);
    int value = 50;
    int *ptr = &value;
    printf("%d\n", *ptr);
    return 0;
}
```